جـامعة بيروت العربية
BEIRUT ARAB UNIVERSITY

Faculty of Science

Department of Mathematics and Computer Science

Artificial Intelligence (CMPS 453)

[DUAL SNAKE AI BATTLE]

Student Name:	**Student ID:**
[JIHANE LAGHA]	[202402818]
[ABIR MELHEM]	[202403504]

Supervisor:

[NADER BAKIR]	Principal Supervisor
[ELIAS EL KHOURY]	Co-supervisor

December 05, 2025

Table of Contents

1. Project Overview

1.1 Objective

The goal of this project is to create an interactive snake game to compare the behavior and performance of two pathfinding algorithms: **A*** and **Greedy Best-First Search**. The game allows observation of how each algorithm navigates dynamic environments and obstacles.

1.2 Key Features

Our game includes several interesting features:

- **Two AI snakes**: One uses A* pathfinding (green), the other uses Greedy (blue)
- **Dynamic pathfinding**: Both snakes calculate new paths every frame as the game changes
- **Manual food placement**: We control where food appears by clicking on the grid
- **Score-based battles**: When snakes collide, the one with more points survives
- **Wrap-around movement**: snakes continue from the opposite edge when hitting walls
- **Maze obstacles**: We added maze patterns to make pathfinding more challenging

1.3 Technology Stack

- **Language**: Java
- **GUI Framework**: Java Swing
- **Architecture**: Model-View-Controller (MVC) pattern
- **Data Structures**: Priority Queues, Hash Maps, Linked Lists, Sets

2. System Architecture

2.1 Core Components

GameController - The brain of the game

- Manages the game loop and updates snakes each frame
- Instructs AI to calculate paths

- Handles collision detection

Snake - Each snake entity

- Stores the snake's body as a chain of positions
- Handles movement and growing after eating
- Wraps around screen edges
- Color-coded: green for A*, blue for Greedy

AIPathfinder - The intelligence

- Contains both A* and Greedy algorithms
- Finds paths around obstacles (snakes, maze walls)
- Has a backup "safe move" when stuck

Food - The goal

- Placed manually by clicking the grid
- Can't spawn on snakes or maze walls

Maze - The obstacles

- Generates different wall patterns
- Makes the pathfinding challenge more interesting
- Seven different patterns to choose from

3. Algorithm Implementation

3.1 A* Pathfinding Algorithm

Characteristics:

- **Optimality**: Guarantees shortest path
- **Completeness**: Always finds a path if one exists
- **Heuristic Function**: Manhattan distance
- **Cost Function**: f(n) = g(n) + h(n)
    - g(n): Actual cost from start to node n
    - h(n): Estimated cost from node n to goal

Implementation Details:

Algorithm: A* Search
Input: start position, goal position, obstacles
Output: optimal path or null

1. Initialize open set with start node
2. Initialize g_score map with start = 0
3. While open set not empty:
   a. Get node with lowest f_score
   b. If node equals goal, return reconstructed path
   c. Mark node as closed
   d. For each neighbor:
      - Skip if out of bounds or blocked
      - Calculate tentative g_score
      - If better than recorded score:
        * Update parent pointer
        * Update g_score and f_score
        * Add to open set
4. Return null (no path found)

Time Complexity: O(b^d) where b is branching factor, d is depth
Space Complexity: O(b^d) for storing explored nodes

3.2 Greedy Best-First Search Algorithm

Characteristics:

- **Speed**: Faster computation than A*
- **Non-optimal**: May find suboptimal paths
- **Heuristic-driven**: Only considers estimated distance to goal
- **Incomplete**: Can get stuck in local optima

Implementation Details:

Algorithm: Greedy Best-First Search
Input: start position, goal position, obstacles
Output: path (potentially suboptimal) or null

1. Initialize current position = start
2. Initialize visited set with start
3. Initialize path with start

4. While current ≠ goal and steps < max:
   a. Get unvisited neighbors
   b. Remove blocked/invalid neighbors
   c. Sort by Manhattan distance to goal
   d. Move to closest neighbor
   e. Add to path and visited set
5. Return path if goal reached, else null

Time Complexity: O(n) where n is number of cells
Space Complexity: O(n) for visited set

3.3 Performance Comparison

Metric	A*	Greedy
Path Optimality	Almost always optimal	Often good enough
Computation Time	Moderate	Fast
Memory Usage	Higher	Lower
Failure Rate	Low	Moderate
Average Path Length	Shorter	Longer

In open spaces without obstacles, both algorithms perform similarly because the obvious path is the best path. The real difference shows up when there are maze walls or complex snake body positions blocking direct routes.

4. Game Mechanics

4.1 Movement System

Wall Wrapping Logic:

```
if (nextX < 0) nextX = cols - 1;      // Left edge → Right edge
if (nextX >= cols) nextX = 0;         // Right edge → Left edge
if (nextY < 0) nextY = rows - 1;      // Top edge → Bottom edge
if (nextY >= rows) nextY = 0;         // Bottom edge → Top edge
```

This creates a toroidal topology, eliminating wall collisions and enabling continuous movement.

Maze Wall Interaction:

When a snake attempts to move into a maze wall, the move is blocked and the snake remains in its current position. Maze walls do not kill snakes; they serve as navigation obstacles that the AI must pathfind around.

4.2 Collision Detection System

We check for three types of collisions every frame:

1. Self-Collision

- When a snake's head touches its own body
- This kills the snake immediately
- Happens to both snakes during competition
- Also kills the winner if it gets too long

2. Head-to-Head Collision

- When both snake heads land on the same square
- The snake with higher score survives
- If scores are tied, both die

3. Head-to-Body Collision

- When one snake's head hits another snake's body
- Again, higher score wins
- The loser disappears, winner continues playing

Important: Hitting maze walls doesn't kill snakes - they just can't move through them. The AI will find a path around, or the snake stays in place if truly stuck.

4.3 Scoring System

- **Initial Score**: 0 for both snakes
- **Increment**: +1 per food consumed
- **Win Condition**: Higher score at collision
- **Survival Mechanism**: Winner continues until self-collision

4.4 Food Placement

Manual Mode (Default):

We chose manual food placement because:

- It gives us control over testing scenarios
- We can create challenging situations
- We can place food strategically to see algorithm differences

5. Maze System

We added mazes to make the pathfinding challenge more interesting and to better show the differences between algorithms.

5.1 Available Maze Patterns

Seven different patterns:

1. **None (Clear)** - No obstacles, original open game
2. **Vertical Lines** - Vertical barriers with gaps
3. **Horizontal Lines** - Horizontal barriers with gaps
4. **Grid** - Creates a grid pattern with passages
5. **Spiral** - Circular spiral from center
6. **Cross** - Cross-shaped walls in the middle
7. **Rooms** - Room-like structures with doorways
8. **Random** - Randomly placed walls

5.2 Why Mazes Matter

Without mazes, both algorithms look almost identical because:

- There are clear, direct paths to food
- The "optimal" path is obvious
- Greedy's simple approach works fine

With mazes:

- Paths require going around obstacles
- Sometimes you must move away from the goal temporarily
- A* shows its advantage by planning the full route

- Greedy can get stuck or take much longer paths

6. Visualization Features

6.1 Path Rendering

We added visual overlays showing what each algorithm is planning:

- **Light green trail** = A* algorithm's planned path
- **Light blue trail** = Greedy algorithm's planned path
- **Numbers at top** = Path length in steps

This visualization taught us something important: the displayed path changes every frame! The algorithms recalculate constantly as the game state changes, so the snake doesn't follow exactly what you see - it's more like a "current plan" that updates continuously.

6.2 User Interface

Game Grid:

- 35 columns × 30 rows
- Each cell is 20 pixels
- Black background with gray grid lines
- Gray blocks for maze walls

Control Panel:

- **Pause/Resume** - Stop and Continue the game
- **Restart** - Reset everything
- **Maze Pattern** - Choose from 7 maze types
- **Speed Slider** - Adjust from 1-12 moves per second
- **Speed Number** - Type exact speed value

Status Display:

- Green Snake (A*) score and status
- Blue Snake (Greedy) score and status
- Current speed
- Death status when a snake dies

7. Conclusion

This project demonstrates the practical differences between **A*** and **Greedy Best-First Search** in a dynamic snake game environment. Real-time path visualization, collision handling, and maze obstacles create an engaging simulation to observe algorithm behavior and decision-making strategies.